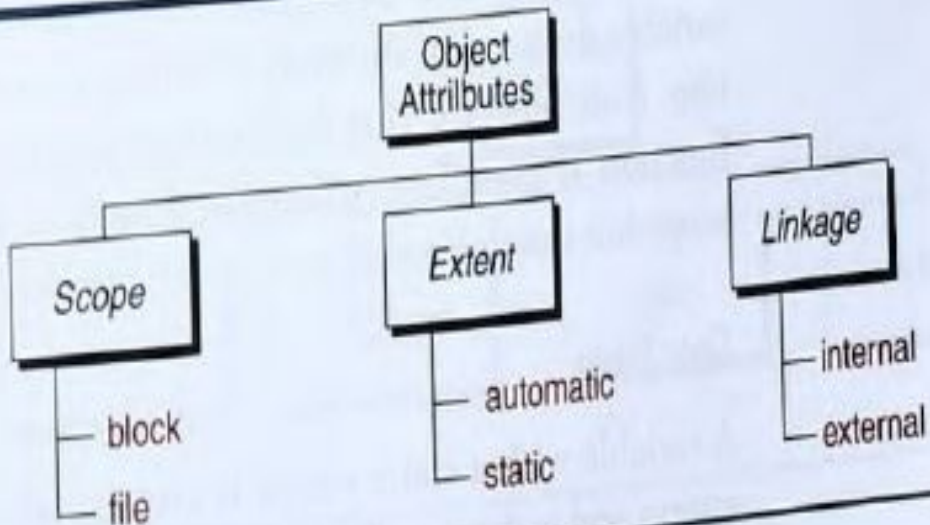


Storage Class

Object Storage Attributes

Storage class specifiers control three attributes of an object's storage as shown in Figure J-1: its scope, extent, and linkage.



1. Scope:- Defines the visibility of an object. It defines where an object can be referenced.
2. Extent:- Lifetime or duration of which computer memory is allocated.
3. Linkage:- Refers to how several modules are linked.

Storage Class Specifiers

C Storage Classes are used to describe the features of a variable/function. These features basically include the scope, visibility, and lifetime which help us to trace the existence of a particular variable during the runtime of a program.

In 'C' a variable can have any one of four Storage Classes.

Auto Variables

Extern Variables

Static Variables

Register Variables

Storage Class specifiers--Auto

Auto storage class is the default storage class for all the local variables. It is created when function is called. When the execution of function is completed, variables are destroyed automatically.

They are also known as local variables because they are local to a function. By default, they are assigned the garbage value by the compiler.

Scope – auto variables are local variables to the function block.

Default value – Garbage value is the default initialized value.

Lifetime – Lifetime of auto variable is bound by the block in which it is defined.

Storage Class specifiers--Auto

```
#include <stdio.h>
int main()
{
    auto int a = 28;    int b = 8;
    printf("The value of auto variable : %d\n", a);
    printf("The sum of auto variable & integer variable : %d", (a+b));
    return 0;
}
```

Output:-

The value of auto variable : 28

The sum of auto variable & integer variable : 36

Storage Class specifiers--Extern

- Variables that are both alive and active throughout entire program are known as extern variables.
- External variables are also known as global variables.
- These variables are defined outside the function.
- These variables are available globally throughout the function execution
- The value of global variables can be modified by the functions.

Scope – They are not bound by any function. They are everywhere in the program i.e. global.

Default value – Default initialized value of global variables are Zero.

Lifetime – Till the end of the execution of the program.

Storage Class specifiers--Extern

```
#include <stdio.h>
extern int x = 32;
int b = 8;
int main() {
    auto int a = 28;
    extern int b;
    printf("The value of auto variable : %d\n ", a);
    printf("The value of extern variables x and b : %d,%d\n ", x, b);
    x = 15;
    printf("The value of modified extern variable x : %d\n", x);
    return 0;
}
```

Output:

The value of auto variable : 28

The value of extern variables x and b : 32,8

The value of modified extern variable x : 15

Storage Class specifiers--Static

- Static variables are initialized only once.
- Compiler persist the variable till the end of the program.
- Static variable can be defined inside or outside the function.

- **Scope** – They are local to the block.
- **Default value** – Default initialized value is Zero.
- **Lifetime** – Till the end of the execution of the program.

Storage Class specifiers--Static

```
#include <stdio.h>
int main() {
    auto int a = -28;
    static int b = 8;
    printf("The value of auto variable : %d\n", a);
    printf("The value of static variable b : %d\n",b);
    if(a!=0)
    printf("The sum of static variable and auto variable : %d\n",(b+a));
    return 0;
}
```

Output:

The value of auto variable : -28

The value of static variable b : 8

The sum of static variable and auto variable : -20

Storage Class specifiers--Register

- Register variables tell the compiler to store the variable in CPU register instead of memory.
- Frequently used variables are kept in registers and they have faster accessibility.
- We can never get the addresses of these variables.
- **Scope** – They are local to the function.
- **Default value** – Default initialized value is the garbage value..
- **Lifetime** – Till the end of the execution of the block in which it is defined.
- Here is an example of register variable in C language
- Syntax: `register int Count`

Storage Class specifiers--Register

```
#include <stdio.h>
int main() {
    register char x = 'S';
    register int a = 10;
    auto int b = 8;
    printf("The value of register variable x: %c\n", x);
    printf("The sum of auto and register variable: %d\n", (a + b));
    return 0;
}
```

Output:

The value of register variable b : S

The sum of auto and register variable : 18

Strings

The string is a sequence of characters terminated with a null character ('\0'). Strings are commonly used for storing and manipulating text in programs.

Syntax: `char string_name[size];`

where size depends on the length of the string.

Ex:- `char str[ ] = "Hello";`

Memory Representation:

Index	0	1	2	3	4	5
Variable	H	e	l	l	o	\0
Address	0x23451	0x23452	0x23453	0x23454	0x23455	0x23456

Initialization of Strings

- In C, string can be initialized in a number of different ways.
- For convenience and ease, both initialization and declaration are done in the same step.

```
char c[] = "abcd";
```

OR,

```
char c[50] = "abcd";
```

OR,

```
char c[] = {'a', 'b', 'c', 'd', '\0'};
```

OR,

```
char c[5] = {'a', 'b', 'c', 'd', '\0'};
```

a	b	c	d	\0
---	---	---	---	----

Strings--Example

Ex:-

```
int main()
{
    char greetings[ ]="Hello World";
    printf("%s", greetings);
}
```

Output:-

Hello World

```
#include <stdio.h>
#include<string.h>
int main()
{
    char str[ ]="Hello";
    str[0]='h';
    printf(" The string is  %s", str);
}
```

Output:-

hello

Strings--Example

```
// C program to illustrate strings
#include <stdio.h>
#include <string.h>
int main()
{
    // declare and initialize string

    char str[] = "program";
    // print string

    printf("%s\n", str);

    int length = 0;

    length = strlen(str);
    printf("Length of string str is %d", length);
    return 0;
}
```

Output:-

program
Length of string str is 7

Read a string input from the user.

// C program to read string from user

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    // declaring string
```

```
    char str[50];
```

```
    // reading string
```

```
        printf("Enter the string");
```

```
    scanf("%s",str);
```

```
    // print string
```

```
    printf("%s",str);
```

```
    return 0;
```

```
}
```

Output:-

1. Hai

Hai

2. Hello

Hello

Note:- Here with the scanf statement & is not used because, As str[] is a character array so using str without braces '[' and ']' will give the base address of this string. That's why we have not used '&' in this case as we are already providing the base address of the string to scanf.

Read a string input from the user.

// C Program to take input string which is separated by

whitespaces

```
#include <stdio.h>
```

```
// driver code
```

```
int main()
```

```
{
```

```
    char str[20];
```

```
    // taking input string
```

```
    scanf("%s", str);
```

```
    // printing the read string
```

```
    printf("%s", str);
```

```
    return 0;
```

```
}
```

Output:-

1. Hai class

Hai

2. Hello students

Hello

Note:- Here we were unable to read the input ,if strings are separated by whitespaces.

So gets and fgets string functions can be used to read the input string which is separated by whitespaces.

Read a string input from the user—using gets() function..

// C Program to take input string which is separated by whitespaces

```
#include <stdio.h>
```

```
// driver code
```

```
int main()
```

```
{
```

```
    char str[20];
```

```
    // taking input string
```

```
    gets(str);
```

```
    // printing the read string
```

```
    printf("%s", str);
```

```
    return 0;
```

```
}
```

Output:-

1. Hai class

Hai class

2. Hello students

Hello students

Read a string input from the user—using fgets() function..

// C Program to take input string which is separated by whitespaces

```
#include <stdio.h>
```

```
// driver code
```

```
int main()
```

```
{
```

```
    char str[20];
```

```
    // taking input string
```

```
        fgets(str, sizeof(str), stdin);
```

```
    // printing the read string
```

```
    printf("%s", str);
```

```
    return 0;
```

```
}
```

Output:-

1. Hai class

Hai class

2. Hello students

Hello students

String—puts() function

The puts() function is used to write a string to the console and it automatically adds a new line character '\n' at the end.

Syntax

```
puts("str");
```

```
#include <stdio.h>
#include<string.h>
int main()
{
    char str[20];
    // taking input string
    fgets(str, sizeof(str),stdin);

    // printing the read string
    puts(str);
    return 0;
}
```

Output:-

Hai class

String—fputs() function

```
#include <stdio.h>
#include<string.h>

// driver code
int main()
{
    char str[20];
    // taking input string
    fgets(str, sizeof(str),stdin);

    // printing the read string
    fputs(str,stdout);

    return 0;
}
```

Output:-

1. Hai class

Hai class

2. Hello students

Hello students

Array of Strings in C

- A string is one dimensional array of characters terminated with null character('\0'), where as array of strings is an two dimensional array of characters terminated with null string('\0').

// C Program to print Array of strings

```
#include <stdio.h>
```

```
// Driver code
```

```
int main()
```

```
{
```

```
char arr[3][10] = {"hai", "fsection", "students"};
```

```
printf("String array Elements are:\n");
```

```
for (int i = 0; i < 3; i++)
```

```
{
```

```
    printf("%s\n", arr[i]);
```

```
}
```

```
return 0;
```

```
}
```

```
String array Elements are:
```

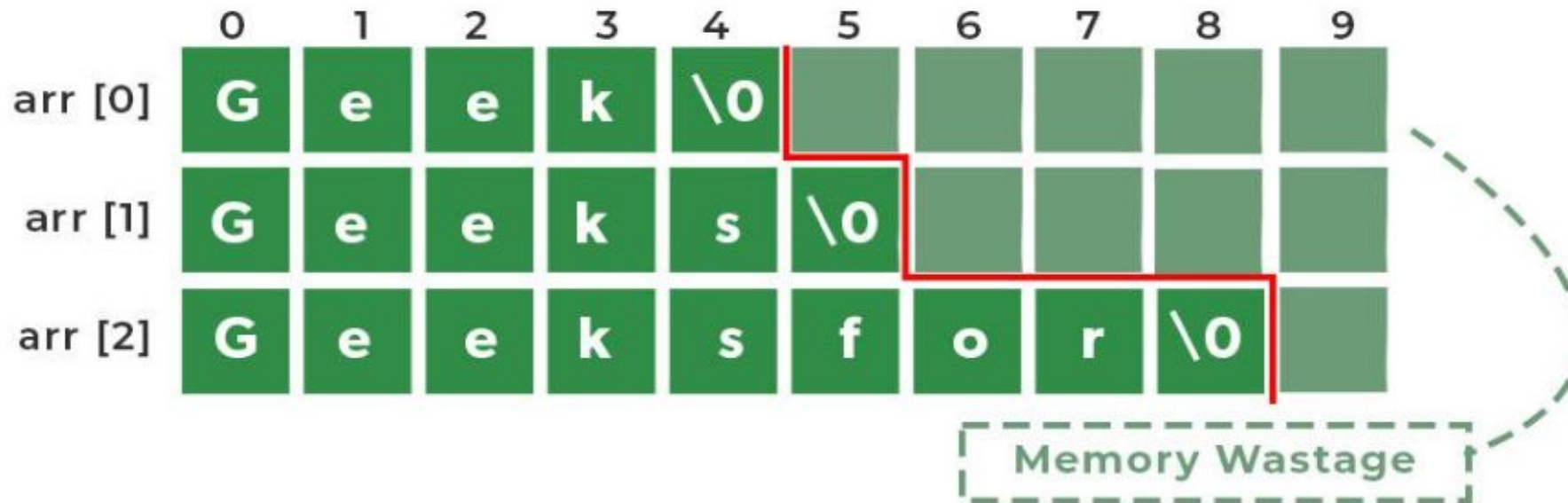
```
hai
```

```
fsection
```

```
students
```

Array of strings

Memory Representation of an Array of Strings



We have 3 rows and 10 columns specified in our Array of String but because of prespecifying, the size of the array of strings the space consumption is high. So, to avoid high space consumption in our program we can use an Array of Pointers in C.

Array of Strings

```
#include <stdio.h>
```

```
// Driver code
```

```
int main()
```

```
{
```

```
char arr[3][10] = {"hai", "fsection", "students"};
```

```
// arr[0]="Hello"; # We cannot directly or assign the values to an array of strings
```

```
    strcpy(arr[0],"Hello"); # Instead strcpy can be used to assign a values to  
array of strings
```

```
printf("String array Elements are:\n");
```

```
for (int i = 0; i < 3; i++)
```

```
{
```

```
    printf("%s\n", arr[i]);
```

```
}
```

```
return 0;
```

```
}
```

Output:-

String array Elements
are:

hello

fsection

students

String Handling/manipulation—built in functions

strcat():Concatenates two Strings

Syntax:- strcat(str1,str2);

Here str2 will be appended at the end of str1. The output would be stored in str1

```
#include<stdio.h>
#include<string.h>
int main()
{
    char str1[50], str2[50];

    printf("Enter first string:\n");
    gets(str1);
    printf("Enter second string:\n");
    gets(str2);
    strcat(str1,str2);
    printf("Concatenated string is: %s", str1);
    return 0;
}
```

Output:-

Enter first string:

Bharath

Enter second String:

Mahan

**Concatenated string
is: Bharath Mahan**

String Handling/manipulation—built in functions

strcmp():-Compares two Strings

Syntax:- `strcmp(str1,str2);`

Here each character of str1 is compared with str2.(Ascii values are considered for comparison). This function returns 0 if two strings are the same otherwise it returns some integer value other than 0.

String Handling/manipulation—built in functions

```
#include<string.h>
int main()
{
    char str1[40], str2[40];
    int d;

    printf("Enter first string:\n");
    gets(str1);
    printf("Enter second string:\n");
    gets(str2);
    d = strcmp(str1, str2);
    if(d==0)
    {
        printf("Given strings are same.");
    }
    else
    {
        printf("Given strings are different.");
    }
    return 0;
}
```

Output:-

Enter first string:

Welcome

Enter second String:

Welcome to C

Given strings are different.

String Handling/manipulation—built in functions

strcpy() :- Copies one String Over another

Syntax:- strcpy(string2,string1);

String handling function strcpy() is used to copy content of one string variable to another string variable i.e. strcpy(string2, string1) copies content of string1 to string2.

```
#include<stdio.h>
#include<string.h>
```

```
int main()
{
    char str1[30], str2[30];

    printf("Enter string:\n");
    gets(str1);
    strcpy(str2, str1);
    printf("Copied string is: %s", str2);
```

```
    return 0;
```

```
} Dept. of CSE(CS), MSRIT
```

Output:-

```
Enter string:
Welcome to C
Copied string is: Welcome to
C
```

String Handling/manipulation—built in functions

Strlen() :- we use strlen() to find the length of the string. String handling function strlen() returns the number of characters in string.

```
#include<stdio.h>
#include

int main()
{
    char str[30];
    int len;

    printf("Enter string:\n");
    gets(str);

    len = strlen(str);

    printf("Length of given string is: %d", len);

    return 0;
}
```

Output:-

Enter string:
Welcome to C
Length of given string is: 12

Pointers

- Pointers are one of the core components of the C programming language.
- A pointer can be used to store the memory address of other variables, functions, or even other pointers.
- The use of pointers allows low-level memory access, dynamic memory allocation, and many other functionality in C.
- A pointer is defined as a derived data type that can store the address of other C variables or a memory location. We can access and manipulate the data stored in that memory location using pointers.

Declartion of Pointers

Syntax of ptr:

`datatype *ptr;`

Where ptr is the name of the pointer.

datatype is the type of data is pointing to

Ex:- `int *ptr;`

- In pointer declaration, we only declare the pointer but do not initialize it. To declare a pointer, we use the (*) dereference operator before its name.
- Here pointer ptr is referring to a random memory location.

Intialization of Pointers

- Pointer initialization is the process where we assign some initial value to the pointer variable.
- We generally use the (&) addressof operator to get the memory address of a variable and then store it in the pointer variable.

Example

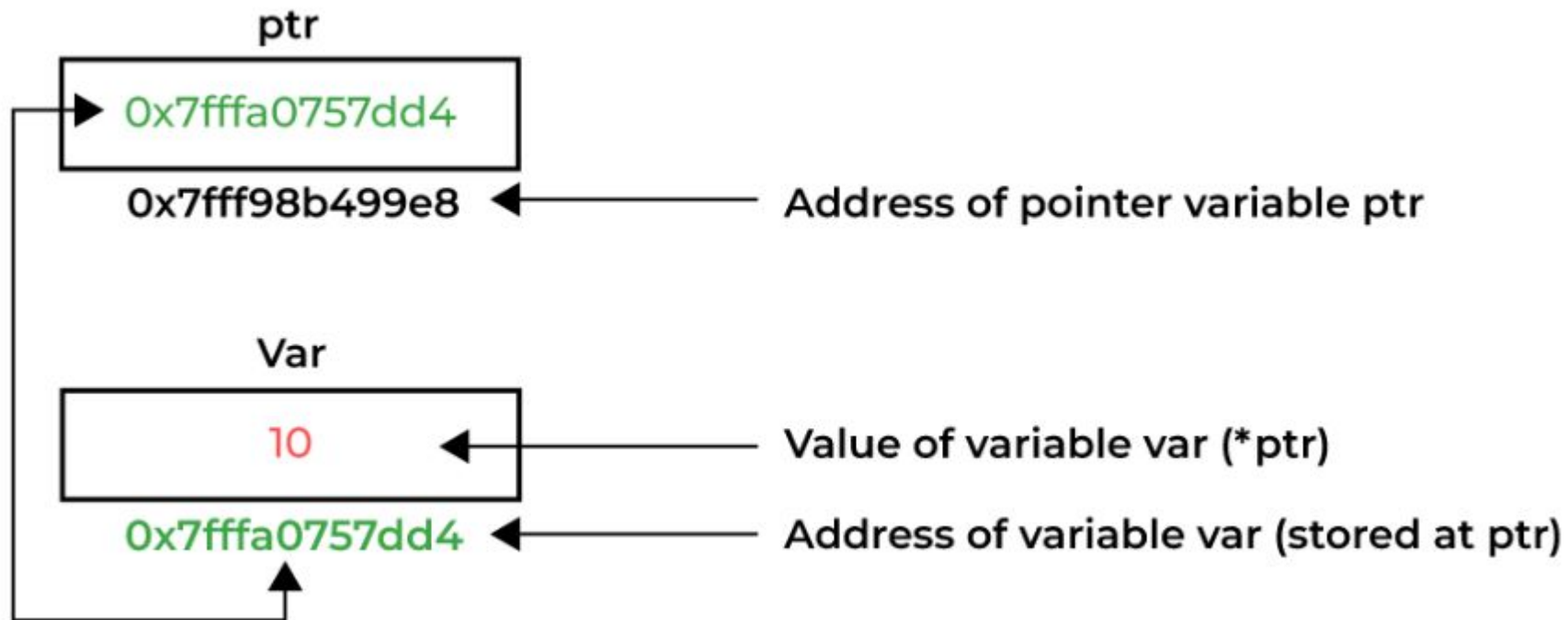
```
int var = 10;  
int * ptr;  
ptr = &var;
```

Or we can also write:

```
int *ptr=&var;
```

Pointer Dereferencing

- Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer.
- We use the same (*) dereferencing operator that we used in the pointer declaration.



Pointer—Example(Accessing value through pointers)

```
#include <stdio.h>

void hello()
{
    int var = 10;
    int* ptr;
    // note that data type of ptr and var must be same
    ptr = &var;
    // assign the address of a variable to a pointer

    printf("Value at ptr = %p \n", ptr);
    printf("Value at var = %d \n", var);
    printf("Value at *ptr = %d \n", *ptr);
}

int main()
{
    hello();
    return 0;
}
```

OUTPUT

```
Value at ptr = 0x7fff1038675c
Value at var = 10
Value at *ptr = 10
```

Passing pointers to a function

- Passing the pointers to the function means the memory location of the variables is passed to the parameters in the function, and then the operations are performed.

1) Arguments passed without a pointer

```
#include <stdio.h>
void swap(int a, int b)
{
    int temp = a;
    a = b;
    b = temp;
}
int main()
{
    int a = 10, b = 20;
    swap(a, b);
    printf("Values after swap function are: %d, %d", a, b);
    return 0;
}
```

- When we pass arguments without pointers the changes made by the function would not be done to the local variables of the function.

- So the output of this swapping program is

Values after swap function are 10 and 20.

Passing pointers to a function

2) Arguments passing with a pointer

```
#include <stdio.h>
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main()
{
    int a = 10, b = 20;
    swap(&a, &b);
    printf("Values after swap function are: %d, %d", a, b);
    return 0;
}
```

- A pointer to a function is passed in this example.
- As an argument, a pointer is passed instead of a variable and its address is passed instead of its value.
- As a result, any change made by the function using the pointer is permanently stored at the address of the passed variable.
- In C, this is referred to as call by reference.

Array of pointers

Pointer to an array: Pointer to an array is also known as array pointer. We are using the pointer to access the components of the array.

```
int a[3] = {3, 4, 5 };  
int *ptr = a;
```

We have a pointer ptr that focuses to the 0th component of the array.

Array of pointers--Example

```
#include <stdio.h>
const int SIZE = 3;
void main()
{
    // creating an array
    int arr[] = { 1, 2, 3 };

    // we can make an integer pointer array to
    // storing the address of array elements
    int i, *ptr[SIZE];

    for (i = 0; i < SIZE; i++) {

        // assigning the address of integer.
        ptr[i] = &arr[i];
    }
    // printing values using pointer
    for (i = 0; i < SIZE; i++) {

        printf("Value of arr[%d] = %d\n", i, *ptr[i]);
    }
}
```

Output:-

Value of arr[0] = 1
Value of arr[1] = 2
Value of arr[2] = 3

Array of pointers

```
// C program to illustrate the use of array of pointers to
// characters

#include <stdio.h>

int main()
{
    char* arr[3] = { "Welcome", "programming", "Students" };

    for (int i = 0; i < 3; i++) {
        printf("%s\n", arr[i]);
    }

    return 0;
}
```

Output:-

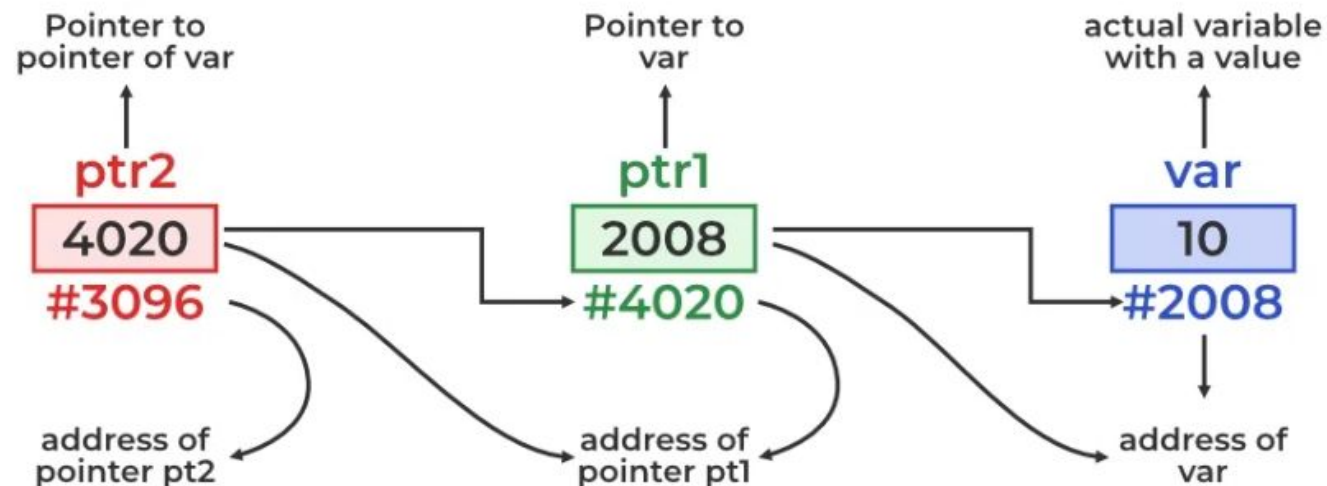
Welcome
programming
Students

Note:- Here there is no memory wastage compared to as in terms of Array of strings (where based on the length of longest string, memory is allocated)

Double Pointer(Pointer to Pointer)

- The pointer to a pointer in C is used when we want to store the address of another pointer.
- The first pointer is used to store the address of the variable.
- And the second pointer is used to store the address of the first pointer. That is why they are also known as double-pointers

Double Pointer



Double Pointer(Pointer to Pointer)

// C program to demonstrate pointer to pointer

#include <stdio.h>

int main()

{

int var = 789;

int* ptr2;

int** ptr1;

ptr2 = &var;

ptr1 = &ptr2;

printf("Value of var = %d\n", var);

printf("Value of var using single pointer = %d\n", *ptr2);

printf("Value of var using double pointer = %d\n", **ptr1);

return 0;

}

Output:-

```
Value of var = 789
```

```
Value of var using single pointer = 789
```

```
Value of var using double pointer = 789
```


Double Pointer(Pointer to Pointer)

```
#include <stdio.h>
```

```
int main() {  
    int x = 10;    // an integer variable  
    int* ptr2 = &x; // ptr2 is a pointer to an integer, holding the address of x  
    int** ptr1 = &ptr2; // ptr1 is a pointer to a pointer, holding the address of ptr2  
  
    // Accessing the value of x through ptr1  
    printf("Value of x: %d\n", **ptr1); // Dereferencing ptr1 twice to get the value of x  
  
    // Accessing the address of x using ptr1  
    printf("Address of x: %p\n", *ptr1); // Dereferencing ptr1 once to get the address of ptr2  
    printf("Address of x using ptr2: %p\n", ptr2); // Should be the same as above  
  
    return 0;  
}
```

Output:-

Value of x: 10

Address of x: 0x7ffdfb6f8c7c

Address of x using ptr2:
0x7ffdfb6f8c7c

Pointer Arithmetic

Pointer Arithmetic is the set of valid arithmetic operations that can be performed on pointers. The pointer variables store the memory address of another variable. It doesn't store any value.

- Increment/Decrement of a Pointer
- Addition of integer to a pointer
- Subtraction of integer to a pointer
- Subtracting two pointers of the same type
- Comparison of pointers

Pointer Arithmetic

Increment/Decrement a pointer:-

If an integer pointer that stores address 1000 is incremented, then it will increment by 4(size of an int), and the new address will point to 1004. While if a float type pointer is incremented then it will increment by 4(size of a float) and the new address will be 1004.

```
#include <stdio.h>
int main()
{
int a = 22;
int *p = &a;
printf("p = %x\n", p);
p++;
printf("p++ = %x\n", p);
p--;
printf("p-- = %x\n", p);
return 0;
}
```

OUTPUT:-

```
p = 2b2b9544
p++ = 2b2b9548
p-- = 2b2b9544
```

Pointer Arithmetic

Addition of Integer to a pointer:-

Consider the same example as above where the ptr is an integer pointer that stores 1000 as an address. If we add integer 5 to it using the expression, $\text{ptr} = \text{ptr} + 5$, then, the final address stored in the ptr will be $\text{ptr} = 1000 + \text{sizeof(int)} * 5 = 1020$.

```
#include <stdio.h>
int main()
{
    int N = 4;
    int *ptr2;
    ptr2 = &N;
    printf("Pointer ptr2 before Addition: ");
    printf("%p \n", ptr2);
    ptr2 = ptr2 + 3;
    printf("Pointer ptr2 after Addition: ");
    printf("%p \n", ptr2);
    return 0;
}
```

OUTPUT:-

Pointer ptr2 before Addition:

0x7ffcd7b26ab4

Pointer ptr2 after Addition:

0x7ffcd7b26ac0

Pointer Arithmetic

Comparison to pointer:-

```
#include <stdio.h>
int main()
{
    int arr[5];
    int* ptr1 = &arr;
    int* ptr2 = &arr[0];

    if (ptr1 == ptr2) {
        printf("Pointer to Array Name and First Element "
               "are Equal.");
    }
    else {
        printf("Pointer to Array Name and First Element "
               "are not Equal.");
    }
    return 0;
}
```

OUTPUT:-

Pointer to Array Name and First Element
are Equal.